

Model Deployment Design Report

API-561, Assignment 2

Problem

In this assignment we consider the problem of how to make a decision-tree-based model available for consumption in a production context. Unlike experimental contexts, serving models to real-world clients requires a scalable, fault-tolerant, and performant solution which we consider here.

Considerations

Our system has a number of functional requirements, largely based on model predictions and the API that exposes same. As we are targeting a production context, we must consider non-functional requirements as well, such as the latency associated with each request, the ability of the system to continue to deliver services if a process crashes, or ensuring the integrity of the system. So, in addition to our functional requirements, which are met by the provided API server and model, the major criteria we measure our system against here are:

- Elasticity: Because we don't know how many clients we'll serve, and because we desire to be efficient with the resources we consume, we should scale our application to match demand
- Stability: While we aim to encode robust logic and test thoroughly prior to release, modern software is an amalgamation of numerous subsystems and discrete components. To increase the likelihood that these parts function as intended in the context of the whole, we should apply regular and continuous pressure along critical performance axes at the earliest opportunity to reduce the likelihood of production failures.
- Reliability: Software systems, particularly as they grow in feature and source-line count, can be intractable to exhaustively test. We will invariably encounter new internal states, particularly given fluctuations in external dependencies or system inputs in a deployment context. We should anticipate this and design our system to be tolerant to component failure where possible.

Design for Production

Modern cloud infrastructure incorporates a host of redundant systems to guarantee quality-of-service across network, compute and storage layers. Recruiting a provider avoids overreliance on any one physical machine or ingress/egress point that on-premises deployment risks.

To achieve elasticity, we recruit the container orchestration system Kubernetes to scale our model-serving application as demand dictates. Employing a load-balancer within the Kubernetes cluster enables us to gracefully route and recover from many types of server crashes or hangs.

While we gain a certain degree of stability from Kubernetes, a single invalid assumption across a software interface could reliably crash our model server. To guard against this type of incompatibility we embrace continuous integration through GitHub actions, and run an application build and regression test suite on every commit. This identifies potentially problematic source-code or incompatibilities as early in the engineering phase as possible and enables early interventions that reduce instability at deployment

time. This same process is optionally used to automate deployments to the Kubernetes cluster on merge to the main branch, effectively automating the operational deployment phase and removing historically problematic and undocumented manual steps.

Finally, to guard against unreliable behavior, we configure Kubernetes to automatically detect and replace hung or failed pods with clean instances. This practice in conjunction with our CI/CD pipelines, improves our overall posture with respect to reliability.

Outcome

The resulting implementation incorporates:

- Google Cloud Platform (GCP) GKS platform to abstract away the complexity of the Kubernetes platform and afford elastic, fault-tolerant, container-based compute;
- the software-as-a-service (SaaS) GitHub platform for a secure, high-availability software development lifecycle management solution;
- the development/operations (DevOps) paradigm for continuous integration, evaluation and deployment.

In totality these choices combine to achieve our stated goals of delivering an elastic, stable and reliable production platform for our decision-tree model and its associated API server.